Universidad Nacional Mayor de San Marcos

Universidad del Perú. Decana de América

Facultad de Ciencias Matemáticas



Sistema de Control de Asistencia para Docentes

Curso: Base de datos

Docente: Ing. Wiler Arturo Ponce Benites

Integrantes:
Roque Huamani, Alejandro Harold
Palomino Gutierres, Joel Raul
Sanchez Ojeda, Nicolas
Bartolo Huere, Edson

22 de junio de 2026

PARTE 1: DESARROLLO DE LA BASE DE DATOS

1. Reglas de Negocio

Las reglas de negocio definen las restricciones y comportamientos lógicos que gobiernan el sistema:

Código	Regla de Negocio	Descripción
RN01	Identificación única del docente	Cada docente es identificado por un código alfanumérico único de exactamente 6 caracteres.
RN02	DNI válido	El DNI del docente debe ser numérico y tener exactamente 8 dígitos.
RN03	Nombres de cursos estandarizados	Los nombres de los cursos se almacenan siempre en MAYÚSCULAS para garantizar uniformidad.
RN04	Código de curso estandarizado	El código del curso debe ser alfanumérico con un máximo de 6 caracteres.
RN05	Créditos válidos	Los créditos de un curso deben estar en el rango permitido: 2, 3, 4, 5 o 6.
RN06	Periodo académico activo	Solo puede existir un periodo académico marcado como activo (`EsActivo = 1`) a la vez.
RN07	Sin superposición de horarios	El sistema no permite asignar a un mismo docente dos clases que se superpongan en día y hora dentro del mismo periodo.
RN08	Horario único por bloque	La combinación de DocenteId + CursoId + DiaSemana + HoraInicio + PeriodoId debe ser única.
RN09	Registro secuencial de asistencia	Un docente debe registrar primero su ENTRADA y cerrar sesión antes de registrar otra en el mismo día.
RN10	Cierre obligatorio antes de nueva sesión	Si un docente tiene dos cursos el mismo día, debe cerrar la sesión del primer curso antes de que el sistema permita registrar el segundo.
RN11	Timeout de seguridad	Las interfaces de registro tienen un temporizador de 20 segundos para cancelar por inactividad.
RN12	Tolerancia de Asistencias	El sistema utiliza límites de tiempo configurables globalmente (en minutos) para calificar las asistencias como ”A tiempo”, ”Tardanza” o ”Falta”.
RN13	Semanas de Exámenes	El sistema contempla semestres de 16 semanas. Las semanas 8 y 16 son exclusivas para exámenes. Si un curso posee horarios de ”Teoría” y ”Práctica”, la sesión de teoría se omite de los requerimientos de asistencia durante esas semanas.

Código	Regla de Negocio	Descripción
RN14	Tipos de Clases	Todo bloque horario debe estar tipificado como "Teoría" o "Práctica" para aplicar correctamente las lógicas de reporte.

2. Identificación del Diccionario de Datos

Tabla: `Docentes`

Almacena la información personal y administrativa de cada docente de la EPCC.

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Id`	INT	NO	PK, IDENTITY	Identificador único.
2	`Codigo`	VARCHAR(20)	NO	UNIQUE, CHECK	Código único del docente.
3	`Nombre`	NVARCHAR(100)	NO	—	Nombres.
4	`Apellido`	NVARCHAR(100)	NO	—	Apellidos.
5	`DNI`	NVARCHAR(20)	SÍ	CHECK	Documento de Identidad.
6	`Clase`	VARCHAR(50)	SÍ	—	Tipo de dedicación docente.
7	`Categoria`	VARCHAR(50)	SÍ	—	Categoría académica.
8	`Departamento`	VARCHAR(100)	SÍ	—	Departamento académico.
9	`Correo`	VARCHAR(100)	SÍ	—	Correo institucional.

Tabla: `Cursos`

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Id`	INT	NO	PK, IDENTITY	Identificador único.
2	`Nombre`	VARCHAR(150)	NO	UNIQUE, CHECK	Nombre oficial del curso.
3	`Codigo`	VARCHAR(20)	SÍ	CHECK	Código según malla.
4	`Creditos`	INT	SÍ	CHECK	Créditos del curso.
5	`Tipo`	VARCHAR(5)	SÍ	—	"O" = Obligatorio, "E" = Electivo.
6	`Escuela`	VARCHAR(100)	SÍ	—	Escuela profesional.
7	`MallaCurricular`	VARCHAR(20)	SÍ	—	Malla vigente.

Tabla: `Periodos`

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Id`	INT	NO	PK, IDENTITY	Identificador único.
2	`Nombre`	VARCHAR(50)	NO	UNIQUE	Nombre descriptivo.
3	`EsActivo`	BIT	NO	DEFAULT 0	Indica si es el vigente.

Tabla: `HorarioClases`

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Id`	INT	NO	PK, IDENTITY	Identificador único.
2	`DocenteId`	INT	NO	FK	Referencia al docente.
3	`CursoId`	INT	NO	FK	Referencia al curso.
4	`DiaSemana`	INT	NO	—	Día de la semana.
5	`HoraInicio`	TIME(7)	NO	—	Hora de inicio.
6	`HoraFin`	TIME(7)	NO	—	Hora de fin.
7	`PeriodoId`	INT	SÍ	FK	Periodo académico.
8	`TipoClase`	VARCHAR(20)	NO	DEFAULT 'TEORÍA'	Especifica si el bloque horario es "TEORÍA" o "PRÁCTICA".

Tabla: `RegistroAsistencia`

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Id`	INT	NO	PK, IDENTITY	Identificador único.
2	`DocenteId`	INT	SÍ	FK	Referencia al docente.
3	`CursoId`	INT	SÍ	FK	Referencia al curso.
4	`FechaHora`	DATETIME	SÍ	—	Fecha/hora de ENTRADA.
5	`HoraSalida`	DATETIME	SÍ	—	Fecha/hora de SALIDA.
6	`PeriodoId`	INT	SÍ	FK	Periodo académico.

Tabla: `ConfiguracionSistema`

Tabla tipo Clave-Valor utilizada para almacenar configuraciones globales e institucionales que rigen el comportamiento de los cálculos y reglas de negocio del sistema de forma persistente.

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Clave`	NVARCHAR(100)	NO	PK	Nombre de la clave de configuración (Ej: ”ToleranciaTardanza”, ”InicioSemestre”).
2	`Valor`	INT	SÍ	—	Valor numérico de la configuración.
3	`Descripcion`	NVARCHAR(255)	SÍ	—	Explicación de la configuración.
4	`ValorTxt`	NVARCHAR(MAX)	SÍ	—	Valores en formato de texto o fecha estructurada.

3. Modelo Lógico (Modelo Entidad-Relación)

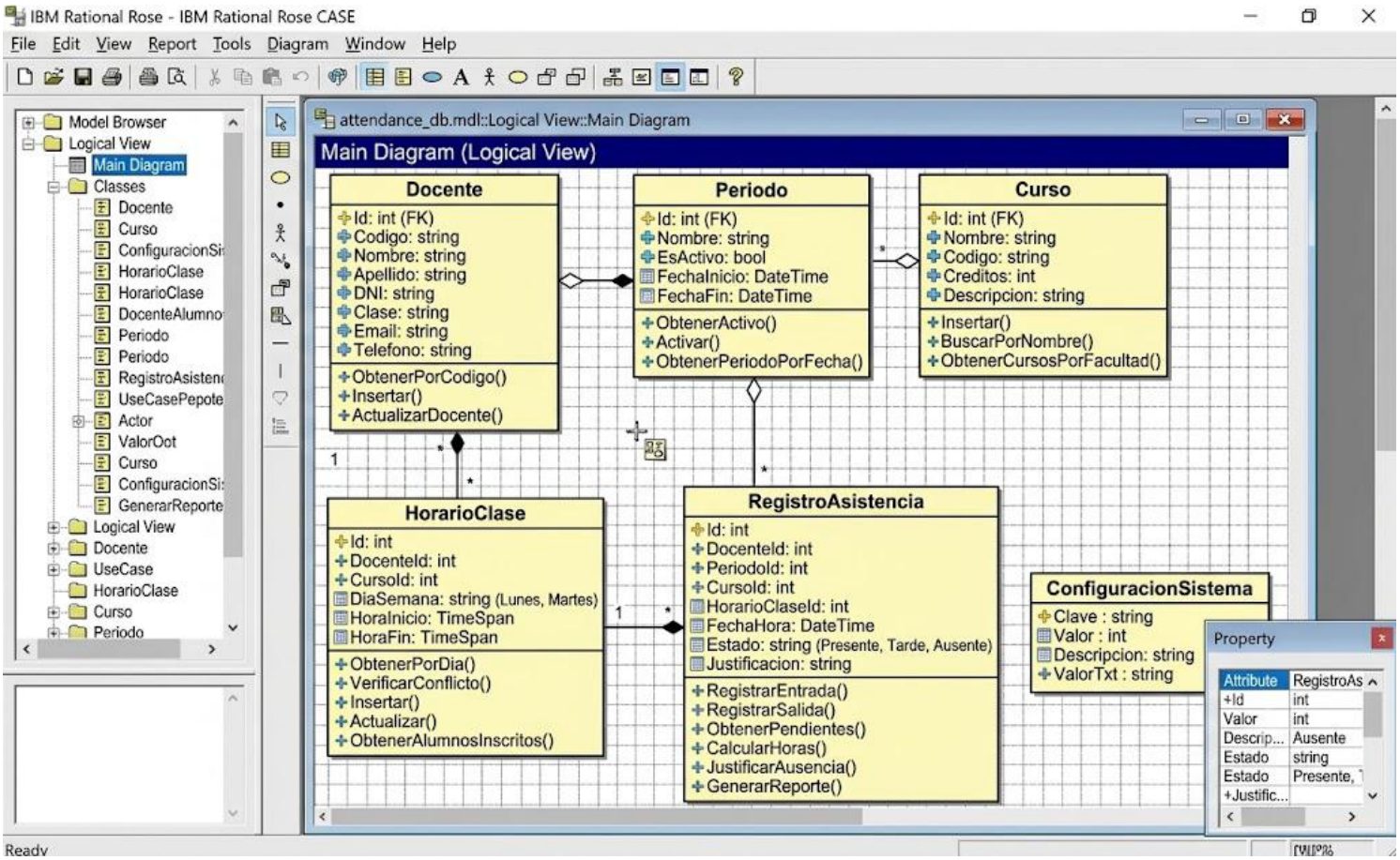


Figura 1: diagrama

Relaciones:

- **Docentes** → **HorarioClases / RegistroAsistencia** (1:N)
- **Cursos** → **HorarioClases / RegistroAsistencia** (1:N)
- **Periodos** → **HorarioClases / RegistroAsistencia** (1:N)
- **ConfiguracionSistema**: Tabla global independiente sin FK, que alimenta reglas globales del negocio a nivel lógico en la capa de negocio (BLL).

4. Scripts

4.1 Script DDL — Creación de la Base de Datos y Tablas

```
-- ============================================================
-- SCRIPT DDL: Creaci n de la Base de Datos AsistenciaFacultad
-- Motor: SQL Server / SQL Server Express
-- ============================================================

CREATE DATABASE AsistenciaFacultad;
GO
USE AsistenciaFacultad;
GO

-- Tabla: Docentes
CREATE TABLE [dbo].[Docentes](
    [Id]            INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    [Codigo]        VARCHAR(20)   NOT NULL,
    [Nombre]        NVARCHAR(100) NOT NULL,
    [Apellido]      NVARCHAR(100) NOT NULL,
    [DNI]           NVARCHAR(20)  NULL,
    [Clase]         VARCHAR(50)   NULL,
    [Categoria]     VARCHAR(50)   NULL,
    [Departamento]  VARCHAR(100)  NULL,
    [Correo]        VARCHAR(100)  NULL
);

-- Tabla: Cursos
CREATE TABLE [dbo].[Cursos](
    [Id]              INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    [Nombre]          VARCHAR(150)  NOT NULL,
    [Codigo]          VARCHAR(20)   NULL,
    [Creditos]        INT           NULL,
    [Tipo]            VARCHAR(5)    NULL,
    [Escuela]         VARCHAR(100)  NULL,
    [MallaCurricular] VARCHAR(20)   NULL
);

-- Tabla: Periodos
CREATE TABLE [dbo].[Periodos](
    [Id]       INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    [Nombre]   VARCHAR(50) NOT NULL UNIQUE,
    [EsActivo] BIT DEFAULT 0
);

-- Tabla: HorarioClases
CREATE TABLE [dbo].[HorarioClases](
    [Id]          INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    [DocenteId]   INT      NOT NULL,
    [CursoId]     INT      NOT NULL,
    [DiaSemana]   INT      NOT NULL,
    [HoraInicio]  TIME(7)  NOT NULL,
    [HoraFin]     TIME(7)  NOT NULL,
    [PeriodoId]   INT      NULL,
    [TipoClase]   VARCHAR(20) DEFAULT 'TEOR A' NOT NULL
);

-- Tabla: RegistroAsistencia
CREATE TABLE [dbo].[RegistroAsistencia](
```

```
    [Id]          INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    [DocenteId]   INT       NULL,
    [FechaHora]   DATETIME  NULL,
    [HoraSalida]  DATETIME  NULL,
    [CursoId]     INT       NULL,
    [PeriodoId]   INT       NULL
);

-- Tabla: ConfiguracionSistema
CREATE TABLE [dbo].[ConfiguracionSistema] (
    [Clave]       NVARCHAR(100) PRIMARY KEY,
    [Valor]       INT           NULL,
    [Descripcion] NVARCHAR(255) NULL,
    [ValorTxt]    NVARCHAR(MAX) NULL
);
```

4.2 Funciones

A continuación se presentan las funciones escalares y de tabla utilizadas en el sistema para simplificar los cálculos de asistencia, tardanzas y la recuperación estructurada de reportes:

```
-- ============================================================
-- FUNCIONES DEL SISTEMA
-- ============================================================

-- 1. Funci n Escalar: Calcular minutos de tardanza
CREATE FUNCTION [dbo].[fn_CalcularMinutosTardanza] (
    @HoraProgramada TIME(7),
    @HoraMarcacion TIME(7)
)
RETURNS INT
AS
BEGIN
    DECLARE @Diferencia INT;
    SET @Diferencia = DATEDIFF(MINUTE, @HoraProgramada, @HoraMarcacion);
    IF @Diferencia < 0
        SET @Diferencia = 0;
    RETURN @Diferencia;
END;
GO

-- 2. Funci n de Tabla: Obtener resumen consolidado de un docente
CREATE FUNCTION [dbo].[fn_ObtenerResumenAsistenciaDocente] (
    @DocenteId INT,
    @PeriodoId INT
)
RETURNS TABLE
AS
RETURN (
    SELECT
        d.Codigo AS DocenteCodigo,
        d.Nombre + ' ' + d.Apellido AS DocenteNombre,
        c.Codigo AS CursoCodigo,
        c.Nombre AS CursoNombre,
        COUNT(ra.Id) AS TotalAsistencias,
        SUM(CASE
            WHEN [dbo].[fn_CalcularMinutosTardanza](hc.HoraInicio, CAST(
                ra.FechaHora AS TIME)) >
```

```
                ISNULL((SELECT Valor FROM ConfiguracionSistema WHERE
                    Clave = 'ToleranciaTardanza'), 15)
            THEN 1 ELSE 0 END) AS TotalTardanzas
    FROM RegistroAsistencia ra
    INNER JOIN Docentes d ON ra.DocenteId = d.Id
    INNER JOIN Cursos c ON ra.CursoId = c.Id
    INNER JOIN HorarioClases hc ON hc.DocenteId = d.Id AND hc.CursoId =
        c.Id AND hc.PeriodoId = ra.PeriodoId
    WHERE ra.DocenteId = @DocenteId AND ra.PeriodoId = @PeriodoId
    GROUP BY d.Codigo, d.Nombre, d.Apellido, c.Codigo, c.Nombre
);
GO
```

4.3 Cursores

El siguiente cursor se utiliza para generar reportes analíticos masivos en consola o para procesos batch internos de auditoría. Permite recorrer cada docente y calcular de forma interactiva sus métricas de asistencia:

```
-- ============================================================
-- CURSOR: Reporte Consolidado de Asistencia y Tardanzas
-- ============================================================

DECLARE @DocenteId INT;
DECLARE @Codigo VARCHAR(20);
DECLARE @Nombre NVARCHAR(100);
DECLARE @Apellido NVARCHAR(100);
DECLARE @TotalClases INT;
DECLARE @TotalTardanzas INT;

-- Declaraci n del cursor para recorrer todos los docentes
DECLARE docente_cursor CURSOR FOR
SELECT Id, Codigo, Nombre, Apellido FROM Docentes;

OPEN docente_cursor;

FETCH NEXT FROM docente_cursor INTO @DocenteId, @Codigo, @Nombre,
    @Apellido;

PRINT '--- REPORTE CONSOLIDADO DE ASISTENCIAS ---';

WHILE @@FETCH_STATUS = 0
BEGIN
    -- Calcular asistencias y tardanzas para el docente actual en el
        periodo activo
    SELECT
        @TotalClases = COUNT(ra.Id),
        @TotalTardanzas = SUM(CASE
            WHEN [dbo].[fn_CalcularMinutosTardanza](hc.HoraInicio, CAST(
                ra.FechaHora AS TIME)) >
                  ISNULL((SELECT Valor FROM ConfiguracionSistema WHERE
                    Clave = 'ToleranciaTardanza'), 15)
            THEN 1 ELSE 0 END)
    FROM RegistroAsistencia ra
    INNER JOIN HorarioClases hc ON ra.DocenteId = hc.DocenteId AND ra.
        CursoId = hc.CursoId
    WHERE ra.DocenteId = @DocenteId
      AND ra.PeriodoId = (SELECT Id FROM Periodos WHERE EsActivo = 1);
```

```
    -- Imprimir la fila procesada
    PRINT 'Docente: ' + @Codigo + ' - ' + @Nombre + ' ' + @Apellido +
          ' | Clases Asistidas: ' + CAST(ISNULL(@TotalClases, 0) AS
             VARCHAR(10)) +
          ' | Tardanzas: ' + CAST(ISNULL(@TotalTardanzas, 0) AS VARCHAR
             (10));

    FETCH NEXT FROM docente_cursor INTO @DocenteId, @Codigo, @Nombre,
       @Apellido;
END;

CLOSE docente_cursor;
DEALLOCATE docente_cursor;
GO
```

4.4 Procedimientos Almacenados

A continuación se listan los procedimientos almacenados principales, los cuales encapsulan la lógica transaccional de registro de entrada/salida y la validación de reglas de negocio directo en la base de datos:

```
-- ============================================================
-- PROCEDIMIENTOS ALMACENADOS
-- ============================================================

-- 1. Procedimiento para registrar la ENTRADA de un docente
CREATE PROCEDURE [dbo].[sp_RegistrarEntradaDocente]
    @DocenteCodigo VARCHAR(20),
    @CursoCodigo VARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @DocenteId INT;
    DECLARE @CursoId INT;
    DECLARE @PeriodoId INT;
    DECLARE @FechaHoraActual DATETIME = GETDATE();

    -- Validar existencia del docente
    SELECT @DocenteId = Id FROM Docentes WHERE Codigo = @DocenteCodigo;
    IF @DocenteId IS NULL
    BEGIN
        RAISERROR('El c digo de docente no existe.', 16, 1);
        RETURN;
    END

    -- Validar existencia del curso
    SELECT @CursoId = Id FROM Cursos WHERE Codigo = @CursoCodigo;
    IF @CursoId IS NULL
    BEGIN
        RAISERROR('El c digo de curso no existe.', 16, 1);
        RETURN;
    END

    -- Obtener periodo activo
    SELECT @PeriodoId = Id FROM Periodos WHERE EsActivo = 1;
    IF @PeriodoId IS NULL
```

```
    BEGIN
        RAISERROR('No existe un periodo acad mico activo.', 16, 1);
        RETURN;
    END

    -- Validar si ya registr  entrada hoy sin registrar salida para ese
        curso
    IF EXISTS (
        SELECT 1 FROM RegistroAsistencia
        WHERE DocenteId = @DocenteId
          AND CursoId = @CursoId
          AND PeriodoId = @PeriodoId
          AND CAST(FechaHora AS DATE) = CAST(@FechaHoraActual AS DATE)
          AND HoraSalida IS NULL
    )
    BEGIN
        RAISERROR('El docente ya tiene una entrada abierta para este
            curso hoy.', 16, 1);
        RETURN;
    END

    -- Registrar Entrada
    INSERT INTO RegistroAsistencia (DocenteId, CursoId, PeriodoId,
        FechaHora, HoraSalida)
    VALUES (@DocenteId, @CursoId, @PeriodoId, @FechaHoraActual, NULL);

    PRINT 'Entrada registrada exitosamente.';
END;
GO

-- 2. Procedimiento para registrar la SALIDA de un docente
CREATE PROCEDURE [dbo].[sp_RegistrarSalidaDocente]
    @DocenteCodigo VARCHAR(20),
    @CursoCodigo VARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @DocenteId INT;
    DECLARE @CursoId INT;
    DECLARE @PeriodoId INT;
    DECLARE @FechaHoraActual DATETIME = GETDATE();
    DECLARE @RegistroId INT;

    -- Validar existencia de docente y curso
    SELECT @DocenteId = Id FROM Docentes WHERE Codigo = @DocenteCodigo;
    SELECT @CursoId = Id FROM Cursos WHERE Codigo = @CursoCodigo;
    SELECT @PeriodoId = Id FROM Periodos WHERE EsActivo = 1;

    -- Buscar la entrada abierta de hoy
    SELECT @RegistroId = Id
    FROM RegistroAsistencia
    WHERE DocenteId = @DocenteId
      AND CursoId = @CursoId
      AND PeriodoId = @PeriodoId
      AND CAST(FechaHora AS DATE) = CAST(@FechaHoraActual AS DATE)
      AND HoraSalida IS NULL;
```

```
    IF @RegistroId IS NULL
    BEGIN
        RAISERROR('No se encontr  una entrada abierta para registrar la
            salida hoy.', 16, 1);
        RETURN;
    END

    -- Registrar Salida
    UPDATE RegistroAsistencia
    SET HoraSalida = @FechaHoraActual
    WHERE Id = @RegistroId;

    PRINT 'Salida registrada exitosamente.';
END;
GO

-- 3. Procedimiento para listar los cursos programados del docente para
   el d a de hoy
CREATE PROCEDURE [dbo].[sp_ObtenerHorariosDiaDocente]
    @DocenteCodigo VARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @DocenteId INT;
    DECLARE @PeriodoId INT;
    DECLARE @DiaSemanaSQL INT;

    -- Mapear d a de la semana actual al est ndar del sistema (1 =
       Lunes, 7 = Domingo)
    SET @DiaSemanaSQL = ((DATEPART(DW, GETDATE()) + @@DATEFIRST - 2) %
       7) + 1;

    SELECT @DocenteId = Id FROM Docentes WHERE Codigo = @DocenteCodigo;
    SELECT @PeriodoId = Id FROM Periodos WHERE EsActivo = 1;

    SELECT
        c.Codigo AS CursoCodigo,
        c.Nombre AS CursoNombre,
        hc.HoraInicio,
        hc.HoraFin,
        hc.TipoClase
    FROM HorarioClases hc
    INNER JOIN Cursos c ON hc.CursoId = c.Id
    WHERE hc.DocenteId = @DocenteId
      AND hc.PeriodoId = @PeriodoId
      AND hc.DiaSemana = @DiaSemanaSQL
    ORDER BY hc.HoraInicio;
END;
GO
```

PARTE 2: DESARROLLO DEL SOFTWARE

1. Ingeniería de Requisitos

1.1 Requisitos Funcionales (RF)

ID	Requisito Funcional	Descripción
RF01	Identificación del docente	El sistema permite identificarse ingresando su código único alfanumérico.
RF02	Detección automática	El sistema detecta los cursos programados del docente para el día actual y los muestra para registro.
RF03	Configuración de tolerancias	El administrador puede establecer límites de tiempo exactos (en minutos) para categorizar entradas a destiempo como ”Tardanzas” o ”Faltas”, almacenándose en la tabla ConfiguracionSistema.
RF04	Gestión de Semestre	El sistema permite fijar una fecha oficial de ”Inicio de Semestre” para proyectar la duración exacta de las 16 semanas académicas.
RF05	Reportes Condicionales	Generación algorítmica de reportes de 16 semanas para cada docente, omitiendo exigencias de asistencia de ”Teoría” en las semanas 8 y 16 si la asignatura contempla horarios de ”Práctica”.
RF06	Exportación Dinámica	Permite la creación y guardado estructurado en el sistema de archivos (carpetas generadas por docente o módulo) de documentos PDF y archivos CSV conteniendo los reportes de asistencias detallados.
RF07	Gestión CRUD Integral	CRUD completo de tablas Docentes, Cursos y HorarioClases, incorporando ahora el atributo TipoClase (Teoría/Práctica).

1.2 Requisitos No Funcionales (RNF)

ID	Requisito No Funcional	Descripción
RNF01	Exportación PDF y CSV	Uso de librerías como iTextSharp para PDF y generadores de CSV estándar para crear reportes persistentes y fáciles de auditar.
RNF02	Experiencia de Usuario UX	Se integran cuadros de diálogo del explorador de archivos nativo de Windows (`FolderBrowserDialog` y `SaveFileDialog`) para permitir selección precisa de rutas de guardado sin imposiciones hard-coded.
RNF03	Interfaz visual (UI)	Uso de Guna.UI2 para componentes redondeados y tablas de búsqueda unificadas con combos de selección por campo, mejorando la legibilidad.
RNF04	Entorno de Despliegue	El sistema será instalado y operado de manera local en una computadora fija (terminal) dedicada para el registro de asistencia dentro de las instalaciones de la facultad.

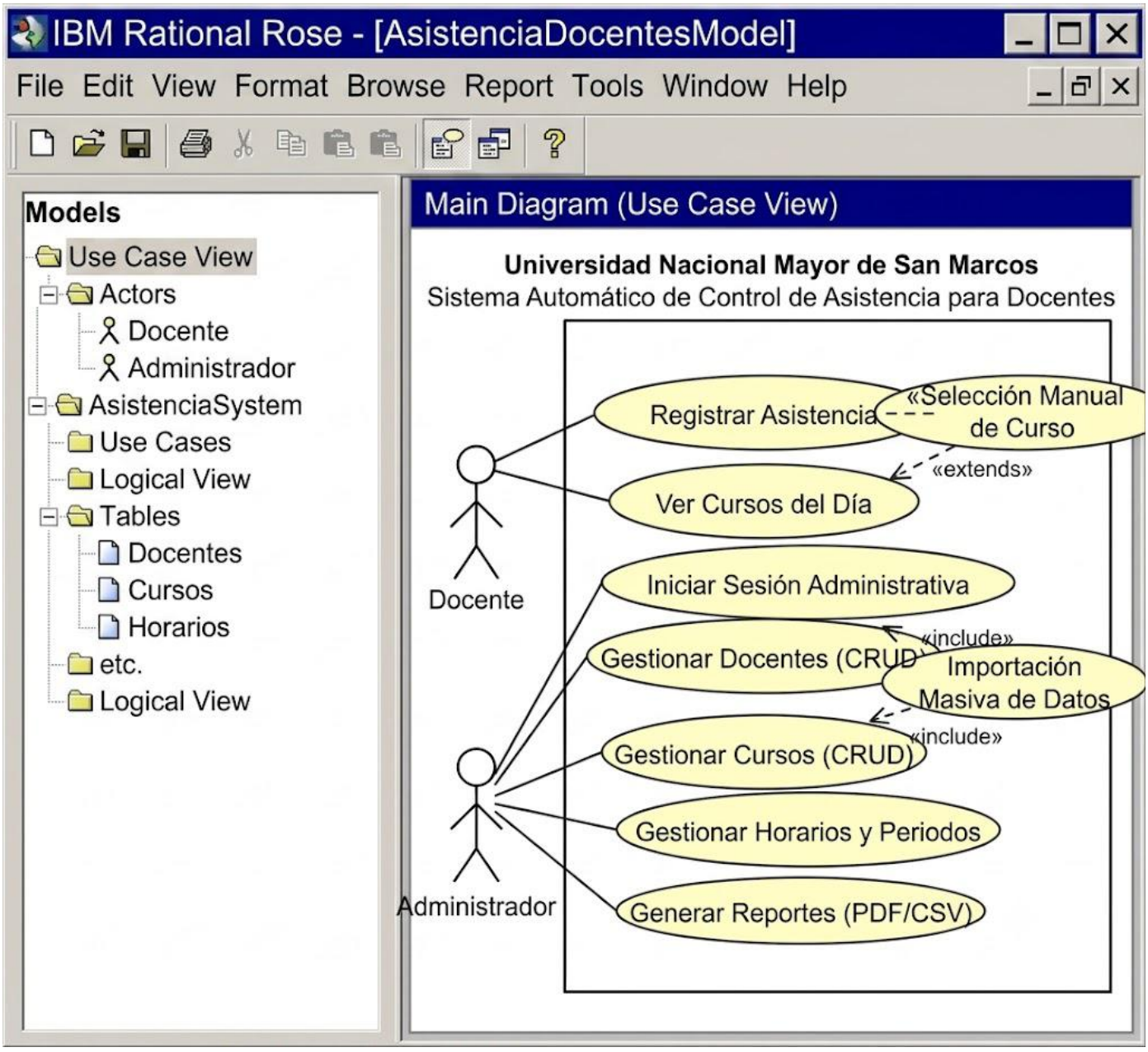
1.3 Lista de Procesos

#	Proceso	Descripción
P01	Registro de asistencia	Flujo principal. Docente ingresa código, sistema detecta cursos y registra entrada/salida calculando tolerancias.
P02	Gestión de docentes	CRUD completo de la tabla Docentes.
P03	Gestión de cursos	CRUD completo de la tabla Cursos.
P04	Gestión de horarios	CRUD de HorarioClases con validación de conflictos e indicación de TipoClase (Teoría/Práctica).
P05	Gestión de periodos	Crear periodos y activar/desactivar.
P06	Generación de reportes	Consultar y exportar a PDF/CSV considerando lógica de semanas de exámenes.
P07	Inicio de sesión	Autenticación de Administrador.
P08	Configurar Límites	Administrador establece tolerancia de tardanzas y faltas.
P09	Configurar Cronograma	Administrador define la fecha de inicio del semestre para proyectar 16 semanas.

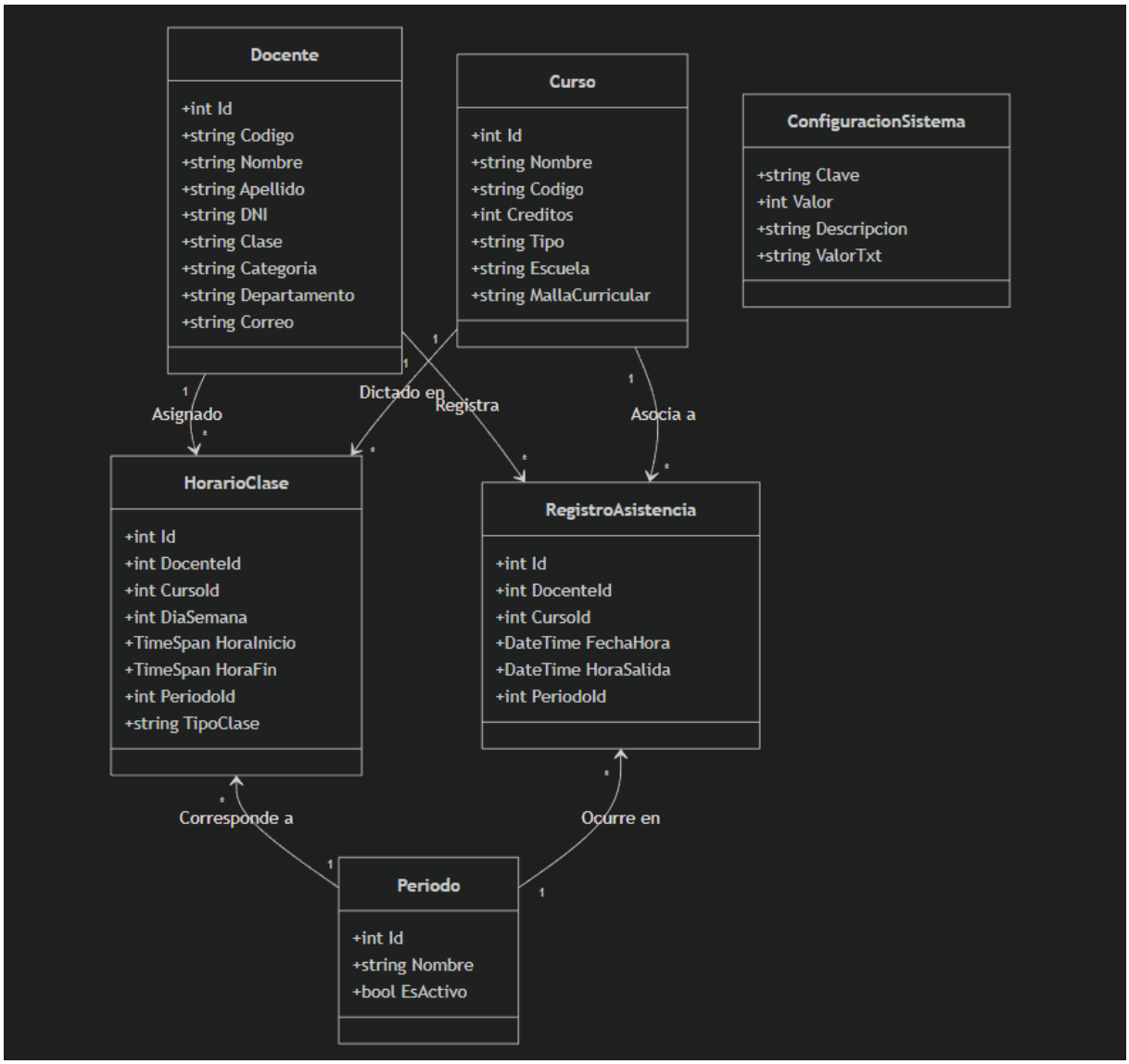
2. Modelado

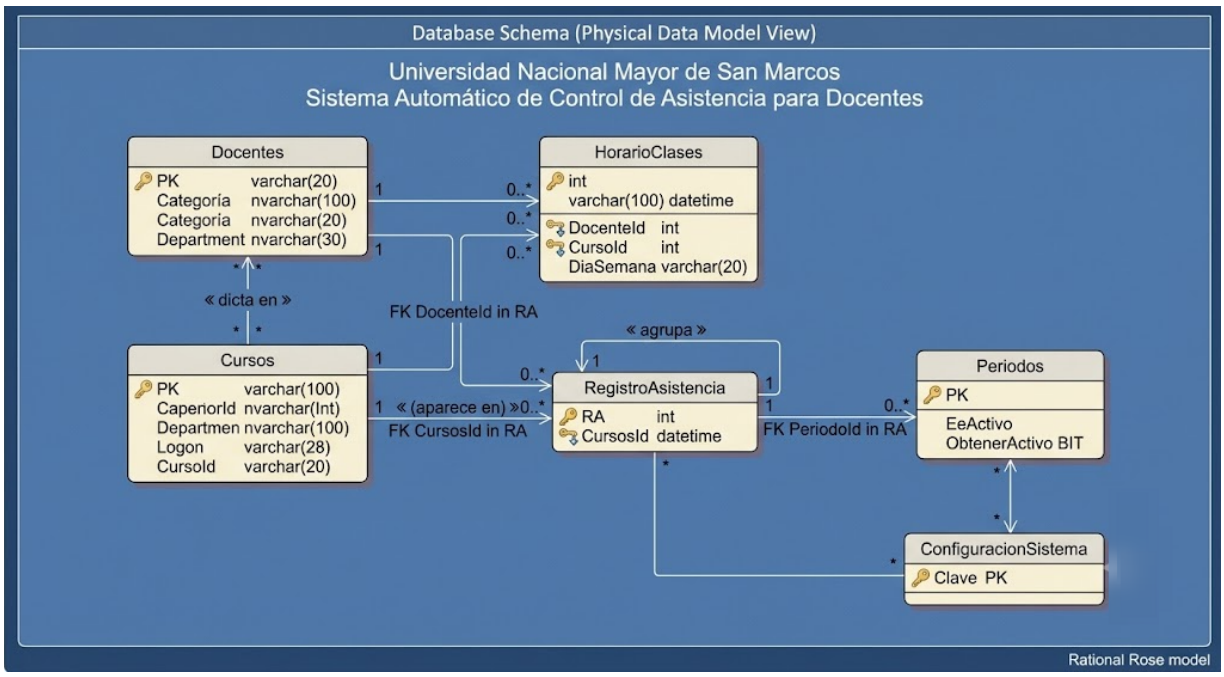
2.1 Diagrama de Casos de Uso

El diagrama visual de Casos de Uso define las interacciones de los actores (Docente y Administrador) con el sistema. (Exportar imagen desde Markdown e insertar aquí).



2.2 Diagrama de Clases





3. Construcción — Arquitectura de 4 Capas

El software se implementó bajo una arquitectura lógica de 4 capas:

- **Capa de Entidades (Entities)**: Define la representación en memoria. Contiene las propiedades que mapean directamente a las columnas de la BD.
- **Capa de Acceso a Datos (DAL)**: Ejecuta sentencias SQL mediante `System.Data.SqlClient`.
- **Capa de Lógica de Negocio (BLL)**: Implementa reglas complejas como determinar tardanzas/faltas leyendo en tiempo real `ConfiguracionSistema`, y calcular si transcurre la semana 8 o 16 para omitir horas de teoría en base a la fecha de inicio del semestre.
- **Capa de Presentación (UI)**: Uso de `Guna.UI2` WinForms. Se incorporan los nuevos `FormCronograma` y `FormConfigLimites` con control de validación visual.

4. Pruebas — Batería de Datos de Prueba

Los casos de prueba se han rediseñado y expandido en formato tabular utilizando los datos reales de carga masiva provenientes de las plantillas CSV (por ejemplo, DOC001, DOC004, INF101, etc.).

ID	Caso de Prueba	Precondición / Entrada	Salida Esperada	Estado
CP-01	Ingreso a Tiempo	**Docente:** DOC001 (A. Turing) **Curso:** INF101 (Lunes, 08:00) **Acción:** Marca asistencia a las 08:05.	Sistema registra entrada. Reporte categoriza como .ª tiempo".	[PASÓ]
CP-02	Límite de Tardanza	**Configuración:** Tolerancia = 15 min. **Docente:** DOC004 (A. Einstein) **Curso:** FIS101 (Martes, 08:00) **Acción:** Marca asistencia a las 08:16.	Sistema registra la hora, pero el reporte lo categoriza como "Tardanza".	[PASÓ]
CP-03	Límite de Falta	**Configuración:** Falta a los 30 min. **Docente:** DOC002 (A. Lovelace) **Curso:** INF201 (Jueves, 10:00) **Acción:** Marca asistencia a las 10:35.	Sistema rechaza registro o reporte lo categoriza directamente como "Falta".	[PASÓ]
CP-04	Omisión Semana 8 (Teoría)	**Configuración:** Semana 8 de 16. **Docente:** DOC001 (A. Turing) **Curso:** INF101 (TEORÍA). **Acción:** Omite registrar asistencia.	El reporte de 16 semanas ignora la inasistencia. No se penalizan las horas dictadas.	[PASÓ]

ID	Caso de Prueba	Precondición / Entrada	Salida Esperada	Estado
CP-05	Exigencia Semana 8 (Práctica)	**Configuración:** Semana 8 de 16. **Docente:** DOC001 (A. Turing) **Curso:** INF101 (PRÁCTICA). **Acción:** Marca asistencia normal.	Se contabilizan las horas requeridas correctamente en época de exámenes.	[PASÓ]
CP-06	Importación Horarios (TipoClase)	**Acción:** Cargar "Plantilla_Horarios.csv"modificado con comas (,). **Filas:** 20 asignaciones (2 por docente).	La tabla HorarioClases se puebla con 20 registros reconociendo Teoría y Práctica.	[PASÓ]
CP-07	Generación de Reporte PDF	**Filtro:** Todos los docentes. **Acción:** Clic en .ExportarPDF". Selección de Escritorio usando `SaveFileDialog`.	Se crea el PDF estructurado con la sumatoria de horas de las 16 semanas sin errores.	[PASÓ]